1BAIA103

Introduction to AI and Applications

NOTES - MODULE 1

Notes Prepared By:

Mr. Sandeep D Dixit,
Assistant Professor,
Department of Mechatronics,
The Oxford College of Engineering,
Bangalore.

Syllabus

Module-1

Introduction to Artificial Intelligence: Artificial Intelligence, How Does AI Work?, Advantages and Disadvantages of Artificial Intelligence, History of Artificial Intelligence, Types of Artificial Intelligence, Weak AI, Strong AI, Reactive Machines, Limited Memory, Theory of Mind, Self-Awareness, Is Artificial Intelligence Same as Augmented Intelligence and Cognitive Computing, Machine Learning and Deep Learning.

Machine Intelligence: Defining Intelligence, Components of Intelligence, Differences Between Human and Machine Intelligence, Agent and Environment, Search, Uninformed Search Algorithms, Informed Search Algorithms: Pure Heuristic Search, Best-First Search Algorithm (Greedy Search).

Knowledge Representation: Introduction, Knowledge Representation, Knowledge-Based Agent, Types of Knowledge.

Textbook 1: Chapter 1 (1.1-1.5), Chapter 3 (3.1-3.7.2), Chapter 4 (4.1-4.4)

Number of Hours: 08

Textbook:

1. Reema Thareja, Artificial Intelligence: Beyond Classical AI, Pearson Education, 2023.

INTRODUCTION TO THE ARTIFICIAL INTELLIGENCE

Artificial Intelligence

In 1950, Alan Turing, published his seminal work - "Computing Machinery and Intelligence". There he raised questions like "Can machines think?". It marked the birth of AI. He even proposed a "Turing Test", Where a human interrogator would try to distinguish between a computer and human text response.

John McCarthy in 2004, defined AI as the science and engineering of making intelligent machines, especially intelligent computer programs.

Definition of AI in layman's view: AI means the intelligence demonstrated by machines that help them to mimic the actions of humans. AI simulated the natural intelligence in machines that are programmed to learn from experiences, adjust to new inputs and perform human-like tasks.
Definition of AI in researcher's view: AI is a set of algorithms that generates results without having to be explicitly instructed to do so, thereby making machines capable of thinking and acting rationally and humanely.
Definition of AI by NITI Aayog: AI refers to the ability of machines to perform cognitive tasks like thinking, perceiving, learning, problem solving and decision-making.

Ïamous languages used to code AI applications are python, R and Java.

Most AI systems, like chess computers or self-driving cars, mainly rely on deep learning and natural language processing.

Computers are very good at tasks like sorting, calculating, storing, and finding patterns, but they still struggle with human skills like recognizing faces, understanding emotions, and having conversations. AI helps machines learn these human-like abilities.

How does AI work?

AI systems need a large amount of labelled training data. They analyse this data to find correlations and patterns and then use those patterns to make predictions. For example, a chatbot trained on text conversations can learn to talk with humans.

AI programming focuses on three cognitive skills; they are as follows.

1. Learning
2. Reasoning
3. Self-correction

Learning

- AI programs focus on acquiring data and creating rules for turning that data into actionable information.
- Rules, also known as algorithms, provide step-by-step instructions to complete a specific task.

Reasoning

- The success of an AI program depends on choosing the right algorithm to reach the desired outcome.

Self-correction

- AI programs are designed to continually enhance their algorithms to provide the most accurate results.

A case study: Why is AI important?

- AI helps businesses gain insights into their operations that were previously unknown.
- AI-enabled machines can perform many tasks better than humans.
- This capability drives the exploration of new business opportunities.

Example: Uber uses machine learning to predict high-demand areas and deploy drivers in advance.

Advantages and Disadvantages of an AI

Advantages

1. Performs well on tasks that uses detailed data.
2. Takes less time to perform tasks that needs to process huge volumes of data.
3. Generates consistent and accurate results.
4. Can be used 24 X 7.
5. Optimizes tasks by better utilizing resources.
6. Automates complex processes.
7. Minimizes downtime by predicting maintenance needs.
8. Enables companies to produce new products having better quality and speed.

Disadvantages

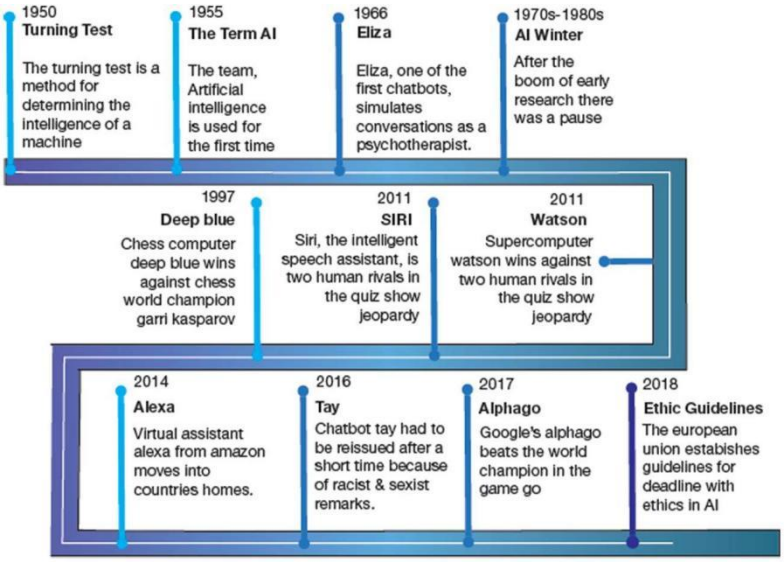
1. Involves more cost.
2. Technical expertise required to develop and use AI applications.
3. Lack of trained professionals.
4. Incomplete or inaccurate data may result in disastrous results.
5. Lacks the capability to generalize tasks

History of AI

1943: McCullough & Pitts proposed the first mathematical model for a neural network.

1950: Turing Test introduced by Alan Turing; Minsky & Edmonds built first neural network computer; Claude Shannon published chess programming paper; Isaac Asimov proposed 'Three Laws of Robotics.'

1952–1954: Arthur Samuel developed self-learning checkers program; IBM translated Russian sentences into English. **1956:** Term 'Artificial Intelligence' coined by John McCarthy; first reasoning program demonstrated by Newell & Simon.



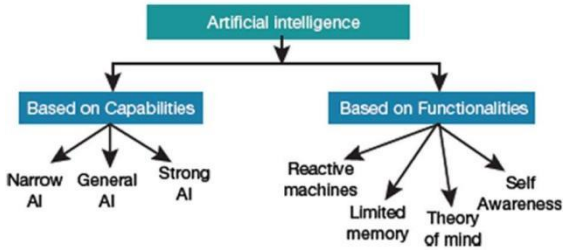
1958–1959: Lisp programming language developed; General Problem Solver (GPS) created; term 'Machine Learning' coined. **1963–1969:** AI Lab at Stanford started; ELIZA, an early chatbot, developed; first expert systems for medical diagnosis created.

1972: Logic programming language PROLOG created.
1974–1980: First AI Winter due to funding cuts.
1980s: DEC created first commercial expert system (R1); Japan launched Fifth Generation Computer Systems project; US funded AI research.
1987–1993: Second AI Winter; FGCS project terminated. **1997:** IBM Deep Blue defeated chess champion Garry Kasparov.

2005–2008: STANLEY, self-driving car, won DARPA Challenge; Google advanced speech recognition.
2011–2012: Apple released Siri; Google Brain project trained neural network on YouTube videos using deep learning.
2014–2015: Google's self-driving car passed driving test; Amazon released Alexa; Baidu's Minwa supercomputer excelled in image recognition.

2016: AlphaGo defeated Go champion Lee Sedol; humanoid robot Sophia created.
2018: Google released NLP engine BERT; Waymo One self-driving taxi service launched.
2020: Baidu's Linear Fold AI predicted SARS-CoV-2 RNA sequence in 27 seconds, aiding vaccine development.

Types of Artificial Intelligence



Weak AI:
It is also known as narrow AI. It is designed to do a specific type of task.

Example: Siri and Alexa are weak AI systems. These systems are already trained with appropriate responses to classify things accordingly. When you instruct Alexa to play a song, it responds by playing that song.

Majority of AI applications that we use today come under this category.
Application of narrow AI has resulted in significant societal benefits. Google search, Image recognition software, self-driving cars and IBM's Watson are some examples of such systems

Strong AI
Strong AI is also known as Artificial General Intelligence, Artificial Super Intelligence or Super Intelligence.
Strong AI makes full attempt to resemble the human brain.
It utilizes cognitive skills and fuzzy logic to perform tasks for which it had not been trained earlier. Such a system needs capability for visual perception, speech recognition, decision-making and translations between languages.

Weak AI	Strong AI
It supports a narrow range of applications with a limited scope.	It supports a wider range of application with a wide scope.
This application is good at specific tasks.	This application has an incredible human-level intelligence.
It uses supervised and unsupervised learning to process data.	It uses clustering and association techniques to process data.
Example: Siri, Alexa.	Example: Advanced robotics

Based on their functionalities AI systems can be classified into four categories:

1. Reactive Machines
2. Limited Memory
3. Theory of mind
4. Self-Awareness

Reactive Machines

- These are very basic machines with no memory to store.
- They just perceive the world and react to it.
- IBM's Deep Blue, which defeated chess grandmaster Kasporov, is an example for reactive machines. Here the AI system just sees the pieces on a chess board and responds without referring to any of its prior experiences.
- As reactive machines lacks capabilities to store past experiences, a reactive machine cannot improve with practice.
- Reactive machines can perform only a limited number of specialized tasks.
- Reactive machines are more trustworthy and reliable as they react the same way to the same stimuli every time.
- Another example for reactive machine: Google's AlphaGo

Limited Memory

- Limited memory machines will retain the data only for short period of time. So that data can be used for only short period of time.
- They cannot keep adding data permanently to a library of their experiences.
- These types of systems are usually used in autonomous vehicles since they need not store data (like recent speed of nearby cars, distance between cars, the speed limit, etc.) that can help them navigate roads.
- Limited Memory AIs are more complex than reactive AI.
- These systems will continuously train the model to analyse and act wisely on new data.
- It will use feedback system to improve itself.
- 3 major Machine Learning models, that applies limited-memory AI are:

  Reinforcement Learning: It enables continuous learning through repeated trial-and-error to make better predictions.

  Long Short-term Memory (LSTM): Such models use past data to help predict the next item in a sequence. More recent information is given a higher priority for prediction than past data. However, this does not mean that past data is not considered while making predictions.

Evolutionary Generative Adversarial Networks (E-GAN):
The ML model evolves over time to explore new ways of utilizing previous experiences to jump formulate new decisions.

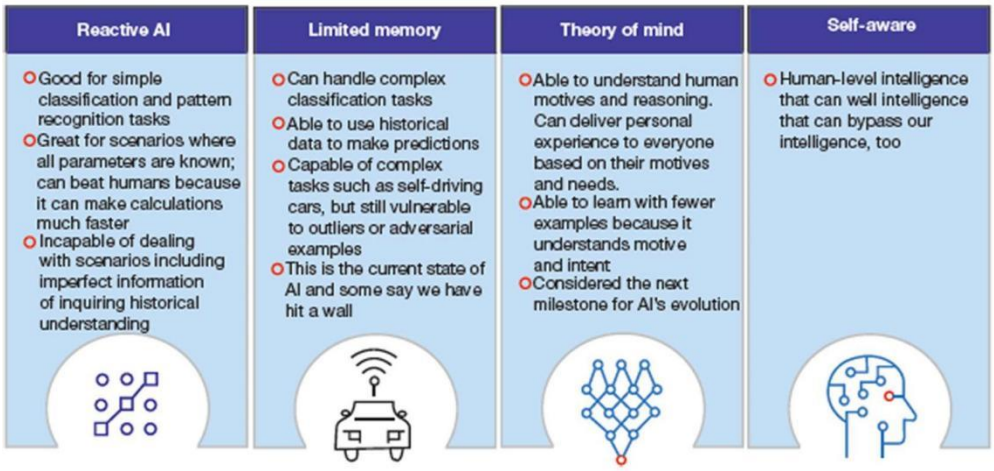
The model is continuously trying to discover a better path.
It makes use of simulations and statistics, to predict outcomes throughout its evolutionary mutation cycle.

Theory of Mind

- The theory of mind focuses on imitating the human brain.
- Such systems are only theoretical as of now but they may become reality very soon.
- These machines must understand thoughts and emotions that affect the behaviour of one's self.

Self-awareness

- After achieving Theory of Mind, the next step for AI is self-awareness.
- Self-aware AI would recognize its own existence and state in the world.
- Such systems could use this awareness to deduce human feelings and needs.
- They would learn from both *what* users say and *how* they say it.
- Contextual understanding would allow them to respond more appropriately.
- These conscious AI systems do not currently exist but may develop in the near future.



Is Artificial Intelligence Same as Augmented Intelligence and Cognitive Computing?

Artificial Intelligence is not same as Augmented Intelligence.
Augmented Intelligence: is weak AI that simply improves products and services. For example, automatically highlighting vital information in business is a part of implementing augmented intelligence.
Artificial Intelligence: or true AI, strong AI or artificial general intelligence, is the future AI that would far surpasses the human brain's ability to understand and implement complex tasks.
Cognitive computing is a term that is especially coined for products and services that mimic that augments human thought processes.

Machine Learning

- **ML** is a branch of computer science that analyses data and identifies patterns to teach a machine to deduce results and make decisions without any human intervention.
- ML algorithms learn from experiences rather than instructions.
- It enables computers (machines) to make data-driven decisions rather than being explicitly programmed for carrying out a certain task.

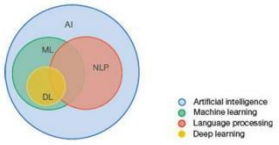


FIGURE 1.4 Relationship between Artificial Intelligence, Machine Learning, Deep Learning and Natural Language Processing

- ML is the subset of AI. Below figure will explain ML as the subset of AI.

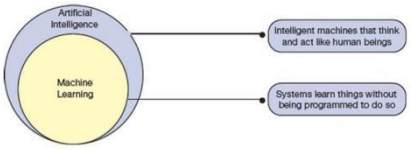
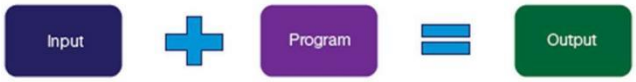


FIGURE 1.6 Difference between AI and ML

Traditional Programming Vs Machine Learning

Traditional Programming

- It involves manually creating a program that accepts data and returns an output.
- Traditional programming uses conventional procedural language like assembly language or a high-level language such as C, C++, Java, JavaScript, Python, etc.



Machine Learning Programming

- It accepts both data as well as output. It then generates the corresponding program for giving the specified output on that input.
- This gives insights that can be used to predict future outcomes.



Traditional Software	Machine Learning
The primary objective is to meet functional and non-functional requirements.	The primary goal is to optimize the performance of the model as every 0.1 % improvement result in significant business value creation.
The quality of the software primary depends on the quality of the code.	The quality of the model depends upon various parameters related to the input data and hyperparameters tuning.
Traditional software is created using traditional programming languages like C, C++, Java.	Machine learning models can be created using different algorithms. Each of these algorithms may perform differently.
As compared to machine learning, few parameters are required.	To get an accurate prediction, a large number of parameters may be required.
Change in data does not change/impact the business functionality. Only new business rules may have to be accommodated.	Machine learning models may have to be retrained with every new data to ensure same or better performance.
Example: Look at the steps given below to convert Celsius scale to Fahrenheit scale Step -1: Take input (Celsius) Step-2: Apply the conversion formula: Fahrenheit = Celsius * 1.8 + 32 Step -3: Print the Output (Fahrenheit) Here, the programmers tell the computer what to do on the input data (multiply Celsius with 1.8 and add 32) to get the result.	Example: Look at the steps given below. Step 1: Feed values in Celsius (i.e., -40, -10, 0, 8, 15, 22, 38) Step -2: Feed corresponding Fahrenheit values (-40, 14, 32, 46, 59, 72, 100) Step -3: Pass these 2 sets of values to Machine Learning (ML) model Step- 4: The ML model computes (convert) the logic to predict any other Fahrenheit value from the given Celsius value. The ML model automatically generates the relationship between Celsius and Fahrenheit to do the conversion.

Deep Learning

- Deep learning is an advanced machine learning technique that passes data inputs through a biologically-inspired multiple layers of neural network.
- So, Deep learning is a subset of ML.
- A deep learning network consists of input layer, number of hidden layers and an output layer.

Features of AI

1. AI is autonomous and can make independent decisions without any human intervention. The best part of an AI system is that they don't have to be programmed manually to respond to every situation. Rather, these systems themselves learn through input data and past experiences.
2. Predict and adapt: AI systems can understand data patterns to make decisions and predictions.
3. An AI system continuously learns from patterns in data.
4. An AI system is said to be reactive as it perceives a problem and acts on perception.
5. Data driven: In the last several years, the cost of technology has gone down considerably. With the availability of cheap data storage (hard disk, etc.), fast processors (CPU, GPU or TPU) and sophisticated deep learning algorithms, huge volumes of data can now be easily extracted. This has led to the rise of data centric AI systems.
6. Data driven AI systems can make accurate predictions based on past experiences have outperformed humans. However, success of these systems depends largely on the availability of correctly labelled large datasets.
7. Finally, an AI system is futuristic since the scope of it being applied in different areas is continuously expanding.

MACHINE INTELLIGENCE

Defining Intelligence:

1. Linguistic Intelligence: The ability to speak and understand language (e.g., narrators, orators).
2. Musical Intelligence: The ability to create and understand music, like recognizing rhythm and pitch (e.g., musicians, singers).
3. Logical-Mathematical Intelligence: The ability to understand abstract concepts and use logic (e.g., mathematicians, scientists).
4. Spatial Intelligence: The ability to visualize and manipulate images in your mind (e.g., map readers, astronauts).
5. Bodily-Kinesthetic Intelligence: The ability to use your body to solve problems or manipulate objects (e.g., dancers, athletes).
6. Intrapersonal Intelligence: The ability to understand your own feelings and motivations (e.g., spiritual leaders, philosophers).
7. Interpersonal Intelligence: The ability to understand other people's feelings and intentions (e.g., mass communicators, interviewers).
8. Artificial Intelligence (AI): A system or machine is said to be artificially intelligent if it can exhibit one or more of these types of intelligence.

Components of Intelligence

There are 5 components of intelligence. They are as follows:

Reasoning
Learning
Problem solving
Perception
Linguistic intelligence

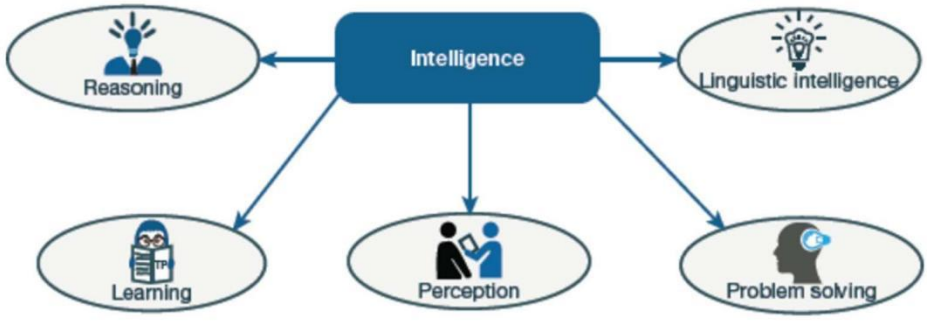


FIGURE 3.1 Components of Intelligence

Reasoning- Reasoning is the process used to make decisions and predictions. It involves analysing information and drawing conclusions based on evidence.
There are two main types of reasoning:
1. Inductive Reasoning: What it is: Making generalizations based on specific observations or examples. How it works: Starts with specific facts or observations and moves to a general conclusion. Example: If you see 10 white swans, you might conclude that all swans are white.

2. Deductive Reasoning: What it is: Drawing a specific conclusion based on general principles or facts. How it works: Starts with a general statement or premise, and moves to a specific conclusion. Example: All swans are birds. A swan is a bird. Therefore, the swan is a bird.

Learning- Learning is the process of gaining knowledge or skills by studying, practicing, or experiencing something. It helps humans, animals, and even AI systems to improve their understanding of different subjects.

There are different types of learning:

1. Auditory Learning: Learning by hearing and listening.
Example: Listening to recorded lectures to understand a concept.

2. Episodic Learning: Learning by remembering events or experiences in a specific order.
Example: Recalling what happened in a sequence, like remembering steps in a recipe.

3. Motor Learning: Learning through physical movement of muscles. Example: Learning how to pick up objects correctly.

4. Observational Learning: Learning by watching and imitating others. Example: Children learn by copying their parents' actions.

5. Perceptual Learning: Learning by recognizing things that have been seen before.
Example: Identifying objects and situations based on prior experiences.

6. Relational Learning: Learning by recognizing patterns in relationships between things.
Example: Adjusting the number of spices in a dish after remembering how much was used last time.

7. Spatial Learning: Learning through visual stimuli like images, maps, and colors. Example: Creating a mental map of a route before actually driving.

8. Stimulus-Response Learning: Learning by reacting to a specific stimulus. Example: Shouting when touching a hot pan, because it causes pain.

Problem-solving- Problem solving is the process of finding a solution to a challenge or issue.

It involves: -
Identifying the problem: Understanding the situation and recognizing the obstacles (either known or unknown).
Making decisions: Choosing the best approach or method to overcome the obstacles and reach the goal.

Perception- Perception is the process of:
Acquiring information through the senses (like sight, hearing, etc.).
Interpreting that information to understand what's happening around us.
Selecting important details and organizing them to form a clear picture.
Humans use sensory organs (like eyes, ears) to perceive the world.
AI systems use sensors (like cameras, microphones) to gather data and understand their environment.

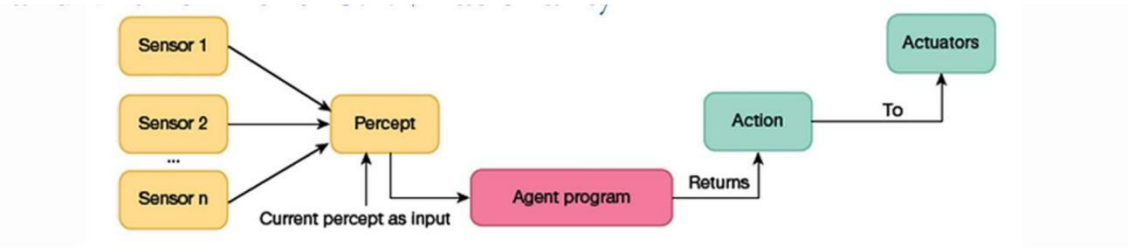
Linguistic intelligence- It is used in in interpersonal communication and defines one's ability to use, comprehend, speak and write the verbal and written language.

DIFFERENCE BETWEEN HUMAN AND MACHINE INTELLIGENCE

AGENTS AND ENVIRONMENT

Agents: AI models who work on specific task in an environment is known as agents.
They perceive the environment using sensors and they act upon the environment using effectors.

Environment: The surroundings in which an agent operates and interacts.



Types of Agents

1. Simple Reflex Agents- choose actions based only on the current percept (the data they receive at a specific moment).

   They are rational only if they make the correct decision based on the current percept.

Working: They use a **condition-action rule** that maps a state (condition) to an action. If the condition is true, the agent performs the action; otherwise, it does nothing.

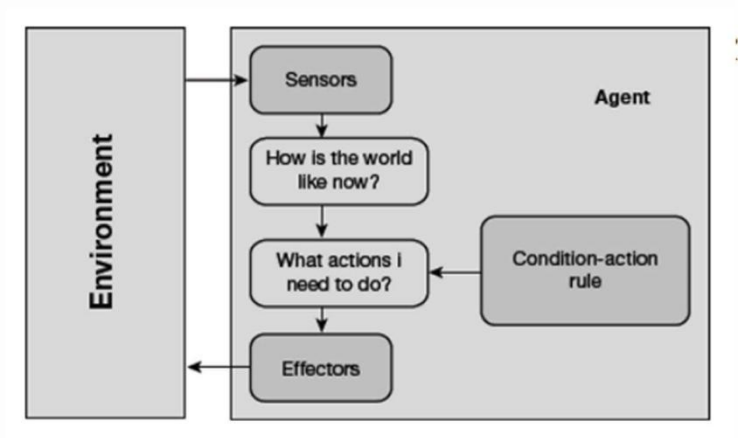
Limitations:

1. They require the environment to be fully observable (the agent must have access to complete information).
2. If the environment is partially observable, the agent might get stuck in infinite loops. In such cases, the agent can only escape the loop if it randomizes its actions.
3. Simple reflex agents have very limited intelligence. If the environment changes, the rules they follow might need to be updated.

2. Model-Based Reflex Agents - use a *model of the world* to choose their actions, which requires them to maintain an internal state. The internal state represents aspects of the current situation that are not directly observable but can be inferred based on the history of percepts.

Working

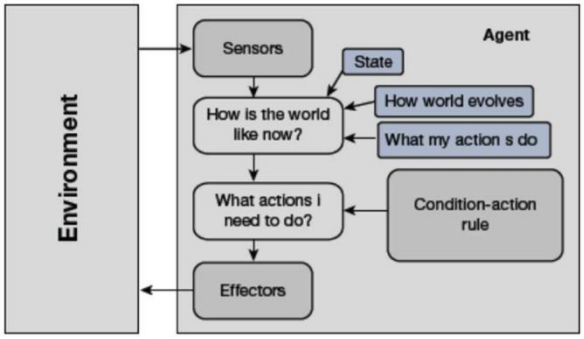
a. The agent updates its internal state by understanding two things:
b. How the world evolves (what happens over time in the environment).
c. How the agent's actions affect the world (the consequences of the agent's actions).



3. Goal-based agents - Goal means a description of desirable situations.

Goal-based agents choose their actions to achieve goals.

This provides more flexibility than reflex agent as the knowledge supporting a decision is explicitly modelled and permits any sort of modifications.

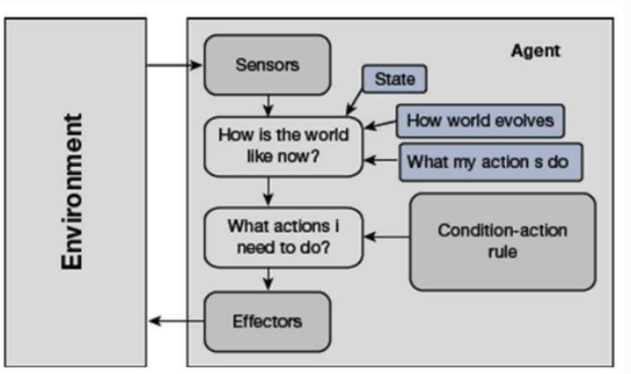


4. Utility-Based Agents - Used when goals are conflicting or difficult to achieve.

   Function: These agents choose actions based on a preference (utility) for each state.

   Goal: They help prioritize goals by choosing actions that lead to the most preferred outcome.

   Flexibility: Provides a way to handle multiple goals by considering their relative importance.

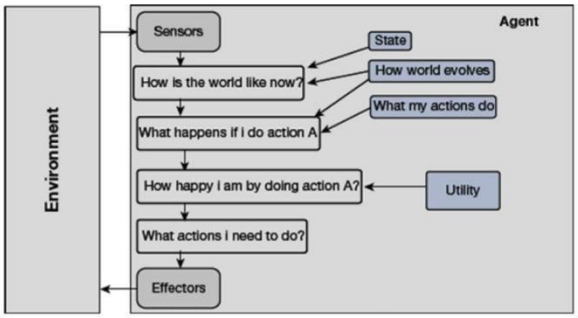


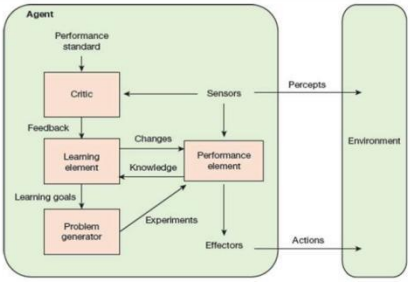
5. **Learning Agent**

   Purpose: A learning agent learns from past experiences and adapts over time. It begins with basic knowledge and gradually improves by learning from its environment.

   Four Main Components:
   1. Learning Element: Responsible for making improvements by learning from the environment.

2. Critic: Provides feedback to the learning element, evaluating how well the agent is performing against a set performance standard.
3. Performance Element: Selects the external action to be taken based on the current situation.
4. Problem Generator: Suggests actions that lead to new experiences for the agent, promoting further learning





Types of environments

1. Discrete/Continuous
a. Discrete: Limited, well-defined states (e.g., chess).
b. Continuous: No limitations on percepts or actions (e.g., self-driving car).

2. Known vs Unknown
a. Known: Agent knows results for all actions.
b. Unknown: Agent must learn how to act (e.g., reinforcement learning).

3. Observable/Partially Observable
a. Observable: Agent can perceive the complete state (e.g., chess).
b. Partially Observable: Agent cannot perceive everything (e.g., Kriegspiel chess).

4. Static/Dynamic
a. Static: Environment does not change while acting (e.g., crossword puzzle).
b. Dynamic: Environment changes during action (e.g., self-driving car).
c. Semi-dynamic: Environment doesn't change, but agent's performance can change.

5. Single Agent/Multiple Agent
a. Single Agent: One agent in the environment (e.g., vacuum cleaner).
b. Multiple Agents: More than one agent, can be competitive or cooperative (e.g., chess, taxi driving).

6. Accessible/Inaccessible
a. Accessible: Agent has full access to environment information (e.g., empty room).
b. Inaccessible: Agent cannot get complete information (e.g., global events). 7.

7. Deterministic/Non-deterministic

a. Deterministic: Next state can be determined from current state (e.g., chess).

b. Non-deterministic: Uncertainty about outcomes (e.g., ludo, dice roll).

8. Episodic/Non-episodic

a. Episodic: Each episode is independent (e.g., simple tasks).

b. Non-episodic: Current actions affect future actions (e.g., long-term decision making).

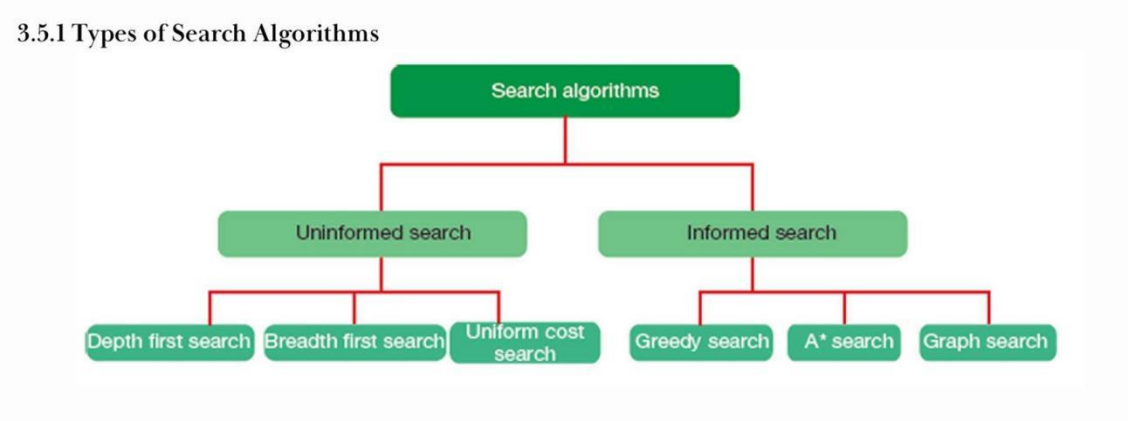
SEARCH

A **search algorithm** is a **step-by-step procedure** used to find a **specific item or solution** among many possible options. AI agents use search algorithms to solve tasks and make decisions. For example, single-player games like Sudoku and tile games use search algorithms to find optimal moves or positions.

1. Uninformed search (or blind search) algorithms have no extra information about the goal state other than what is provided in the problem definition. The algorithm blindly explores the search space without considering how close it is to the goal.
2. Informed search algorithms, also known as heuristic search algorithms, are a class of search algorithms in Artificial Intelligence that utilize problem-specific knowledge to guide the search process more efficiently towards a goal state.

Components of a Search Problem

1. State Space: The set of all possible states the agent can reach.
2. Start State: The initial state where the search begins.
3. Goal Test: A function that checks if the current state is the goal state.
4. Solution: A sequence of actions (plan) that transforms the start state to the goal state, achieved using search algorithms.



Properties of Search algorithms

1. Completeness: A search algorithm is complete if it guarantees at least one solution for a given input.
2. Optimality: A search algorithm is optimal if it provides the best solution with the lowest path cost.

Time Complexity: The amount of time an algorithm takes to complete a task.
Space Complexity: The amount of memory required for the search process. A good search algorithm should use less time and less memory.

Algorithm_1 : Depth First Search Algorithm (DFS)

Depth First Search (DFS) is a simple search algorithm used to explore a tree or graph by starting from the root node and exploring as far as possible along each branch before backtracking.

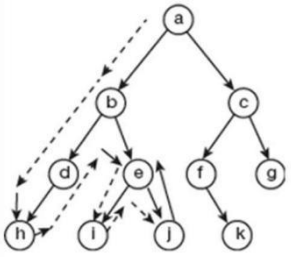
Steps:

1. Start from the root node: Begin searching from the root node (node A).
2. Explore each branch: Move from node A to its child node (B), then to the next child (D), and continue exploring until you reach the leaf node (the last node of that branch).
3. Backtrack: If the key you're looking for isn't found at the leaf node, backtrack to the last node with unexplored branches and explore them.
4. Repeat the process: Continue exploring each branch by backtracking and then moving to the next unexplored branch, until the entire tree is searched or the goal is found.

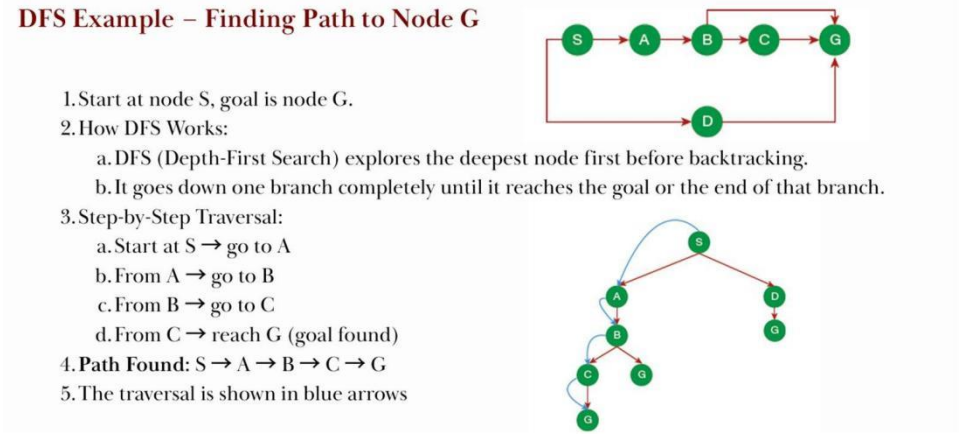
DFS uses a stack (Last-In-First-Out or LIFO structure) to keep track of nodes to visit next.

Root node: a
Goal Node: j

Path found is : a → b → d → h → i → e→ j



Example



Advantages of DFS

1. Uses less Memory.
2. Faster compared to BFS, especially when tree is too deep.

Disadvatges of DFS

1. Recurring states
2. DFS may get stuck in an infinite loop when it keeps going deeper.
3. If number of paths increases, complexity of algoritm also increases.
4. DFS cannot check for duplicate nodes, potentially leading to inefficiency in the search.

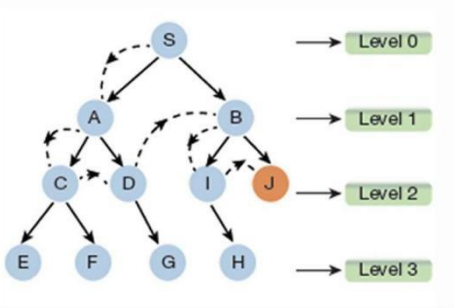
Algorithm_2 Depth Limited Search (DLS)

Depth-Limited Search (DLS) is similar to Depth First Search (DFS), but with a predetermined depth limit to avoid infinite paths. When the search reaches the specified depth limit, nodes at that depth are treated as leaf nodes (i.e., nodes with no successors).

Root Node: S
Goal Node: J

If level = 2

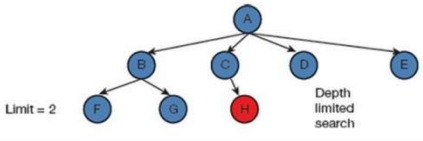
Path is: S → A → C → D → B→ I → J



Example:

DLS Example – Searching for Node H (Limit = 2)

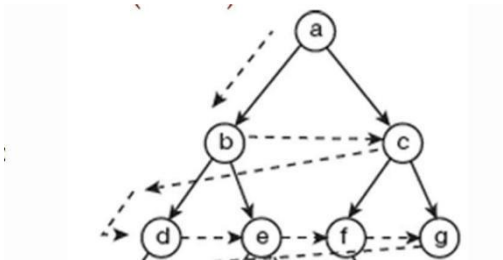
- Search for node H using Depth-Limited Search (DLS) with a depth limit of 2.
- **How DLS Works:**
  - Start at root node A (level 0).
  - Explore all nodes at level 1: B, C, D, E.
  - Move to level 2 and explore children of B → H not found → backtrack.
  - Explore children of C at level 2 → H found.
- **Traversal Path:**
  - A → B → (children of B checked) → backtrack
  - A → C → H → goal found
- **Key Idea:**
  - DLS explores nodes up to a fixed depth limit.
  - If the goal isn't found at the current depth, it backtracks to explore other branches.
  - Useful when the goal depth is known or limited.



Algorithm_3 Breadth First Search (BFS)

Breadth First Search (BFS) is an algorithm that traverses the tree breadthwise, meaning it explores all nodes at the current level before moving to the next level.

Root Node: a



Goal Node: k

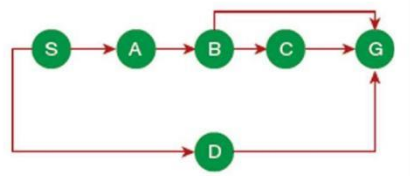
Path: a→ b → c → d → e → f → g → h → i → j → k

Example:

Example: Find the BFS traversal from node S to node G

Find the path from S to G using Breadth-First Search (BFS).

- **How BFS Works:**
  - BFS explores nodes level by level, starting from the root (shallowest nodes first).
  - It explores all nodes at the current depth before moving to the next level.
- **Step-by-Step Traversal:**
  - Start at S → explore children nodes at level 1.
  - Node D is explored first → then G is reached.
- **Path Found:**
  - S → D → G
- **Key Idea:**
  - BFS always finds the shallowest solution (fewest number of nodes in path).
  - Path length = level of the shallowest solution.



Algorithm_4 Uniform Cost Search (UCS)

UCS finds the cheapest path from a start node to a goal node when the step costs are different. It calculates the total cost to reach each node from the start. It always expands the node with the lowest cumulative cost next.

Root node: S Goal node: G

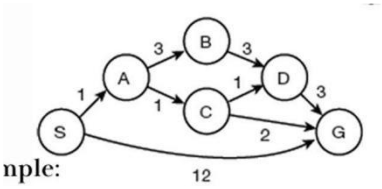
Path: S → A → C → G

Key Features:

UCS always finds the optimal (cheapest) path.

Works like BFS if all step costs are the same.

Usually uses a priority queue, which gives priority to nodes with lowest cumulative cost.

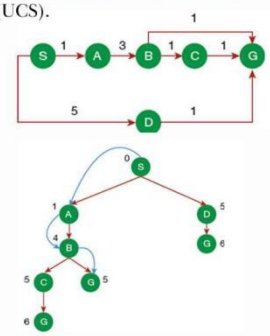


nple:

Example: Find the path and cost to move from node S to node G in the graph given below

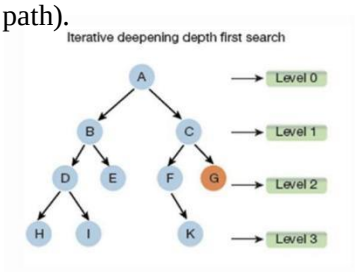
Find the cheapest path from S (start) to G (goal) using Uniform Cost Search (UCS).

- **How UCS Works:**
  - UCS expands nodes based on cumulative cost from the start node.
  - Always selects the node with the lowest total cost to explore next.
- **Step-by-Step Traversal:**
  - Start at S → check child nodes A and other options.
  - Choose A because it has the lowest cumulative cost.
  - From A, explore B → cost is still lowest.
  - From B, reach G → goal reached with least total cost.
- **Path Found:**
  - S → A → B → G
- **Total Cost:**
  - 5
- Key Idea: UCS always finds the optimal path based on actual costs, not depth or heuristic.



Algorithm_5 Iterative Deepening Depth First Search (IDDFS)

IDDFS is used to search for a goal node when the depth of the goal is unknown. It combines the advantages of DFS (low memory use) and BFS (guaranteed shortest path).



Working:

1. Start with a depth limit of 1, perform DFS up to that depth.
2. If the goal isn't found, increase the depth limit by 1 and repeat DFS.
3. Continue increasing the depth limit until the goal is found.

Key Features:

Memory Effeicient, Fast Search, Best suited when search space is large and depth is unkown.

- **Step-by-Step Traversal:**
  - Iteration 1 (Depth 0): Explore node A
  - Iteration 2 (Depth 1): Explore nodes B and C
  - Iteration 3 (Depth 2): Explore nodes D, E, F, G
  - Iteration 4 (Depth 3): Explore node H → goal found

Complexity:

- Time Complexity: O(b^d)
- Space Complexity: O(b·d)
  - Where b = branching factor, d = depth of the goal

Algorithm_6 Bidirectional Search

The search happens from both directions at the same time:

a. Forward Search: From the start node toward the goal.
b. Backward Search: From the goal node toward the start.

The search stops when the two paths meet at a common node.

Advantage:

Reduces the search space because each search only goes half the total distance.

Faster than searching in a single direction.

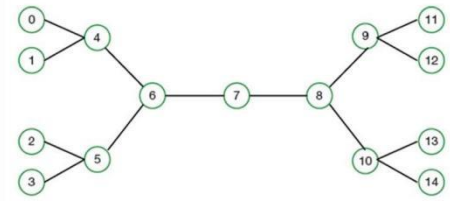
Example:

Example: Consider the graph given below and apply bidirectional search on it to reach goal node 14 from source node 0.

Find the path from source node 0 to goal node 14 using Bidirectional Search.

How Bidirectional Search Works:

- Two searches are run simultaneously:
  - Forward Search: From 0 → 14
  - Backward Search: From 14 → 0
- Both searches continue until they meet at a common node.



Step-by-Step Traversal:

- Forward search: 0 → ... → 7
- Backward search: 14 → ... → 7
- Intersection at node 7 → path found

- **Path Found:**
- **Concatenate forward and backward paths:**
  - **0 → ... → 7 → ... → 14**

Algorithm_7 Pure Heuristic Search

Pure heuristic search is an informed search algorithm that efficiently solves problems involving a large number of possible states by utilizing problem-specific knowledge.

It works by expanding nodes based on their heuristic value h(n), which represents an estimate of the distance to the goal.

The algorithm maintains two lists: an OPEN list for nodes yet to be expanded and a CLOSED list for nodes already expanded.

At each step, the node with the lowest heuristic value is chosen for expansion.

During the process, heuristic values are applied to child nodes, and these nodes are then added to the OPEN list according to their heuristic value. Shorter paths are retained while longer ones are discarded, and the algorithm repeats this procedure until the goal state is reached.

The main benefit of pure heuristic search is that it reduces unnecessary exploration and focuses on the most promising paths, leading to efficient problem-solving.

Examples of this approach include the Greedy Best-First Search and A* Search Algorithm.

Algorithm_8 Greedy Best-First Search

Steps

a. Insert the start node in the OPEN list.
b. If OPEN list is empty, stop (goal not found).
c. Remove the node with lowest heuristic value h(n) from OPEN, add it to CLOSED list.
d. Expand this node and generate its successor nodes.
e. If any successor is the goal, stop and return success.
f. If a successor is not in OPEN or CLOSED, add it to OPEN list.

g. Repeat from Step b until goal is found.

Example

```
text
          S (15)
         /      \
   A (11)        B (5)
               /    \
          E (4)     F (2)
                  /  |  \
              I (3) G (0) J (3)
```

Table shows the heuristic estimates:

Node	h(n)	Node	h(n)	Node	h(n)
A	11	E	4	I, J	3
B	5	F	2	S	15
C, D	9	H	7	G	0

- Step-by-Step Traversal:
  - **Iteration 1:**
  - Start at S.
  - Successors of S: A, B.
  - OPEN = [A, B], CLOSED = [S].
  - B has the lowest h(n) value, so expand B.
  - OPEN = [A], CLOSED = [S, B].

- Step-by-Step Traversal:
  - **Iteration 2:**
  - Generate successors of B: E, F, A.
  - OPEN = [E, F, A], CLOSED = [S, B].
  - F has the lowest h(n) value, so expand F.
  - OPEN = [E, A], CLOSED = [S, B, F].

- **Iteration 3:**
- Generate successors of F: I, G, E, A.
- OPEN = [I, G, E, A], CLOSED = [S, B, F].
- G is the goal, so the algorithm terminates successfully.
- OPEN = [I, E, A], CLOSED = [S, B, F, G].
  -

The path is S → B → F → G

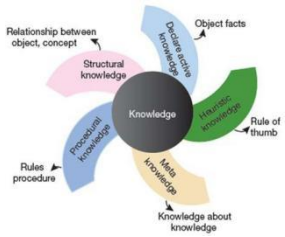
Knowledge

Knowledge is the basic element of our brain as it helps us to know and understand the things logically and is gained by experiences of facts, data and situations.

5- types of knowledge

1. Meta knowledge: is the knowledge about knowledge.
2. Heuristic knowledge: is the knowledge about a specific topic.
3. Procedural knowledge, also known as imperative knowledge, gives information about how to achieve or do something.
4. Declarative knowledge is the information that we have about an object.It is also known as descriptive knowledge.

5. Structural knowledge is the basic knowledge to solve complex problems. It describes relationships between various concepts or objects such as kind of, part of, and grouping of something.



Like humans use their knowledge for decision making, in order to introduce intelligence for machines on eneed to feed the knowledge to machine in a form which is readable for it. This is known as knowledge representation. Types of knowledge we need to represent is as below.

1. Object: Information and facts about all objects (relevant in context). For example, in a self-driving car, vehicles and roads are objects.

2. Events: Information and facts about actions which occur in the real world. For example, in a self-driving car, an event can be applying breaks when an object comes in front of it.

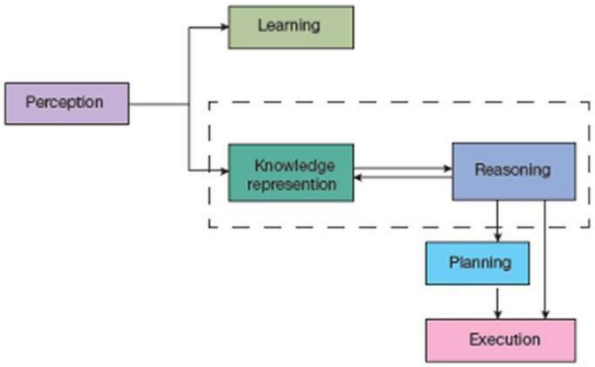
3. Performance: The manner in which actions are performed. It describes the behaviour related to how to do things.

4. Meta-knowledge: It is the knowledge about knowledge (what we know).

5. Facts: Facts are the truths about the real world that needs to be represented for an intelligent agent.

6. Knowledge base: It is the most important component of a knowledge-based agent that stores a group of sentences (technical sentences, not simple English language ones).

Cycles of Knowledge representation



- An AI system processes information through several stages: Perception (collects data), Learning (improves from data), Knowledge Representation & Reasoning (stores and manipulates knowledge), Planning, and Execution.
- Effective knowledge representation must be accurate, allow new inferences, operate efficiently, and easily acquire new knowledge.
- The core AI intelligence lies in how knowledge is represented and reasoned with, using structured techniques like logical and semantic representations.

Knowledge Representation Requirements

A good knowledge representation system must have the following properties.

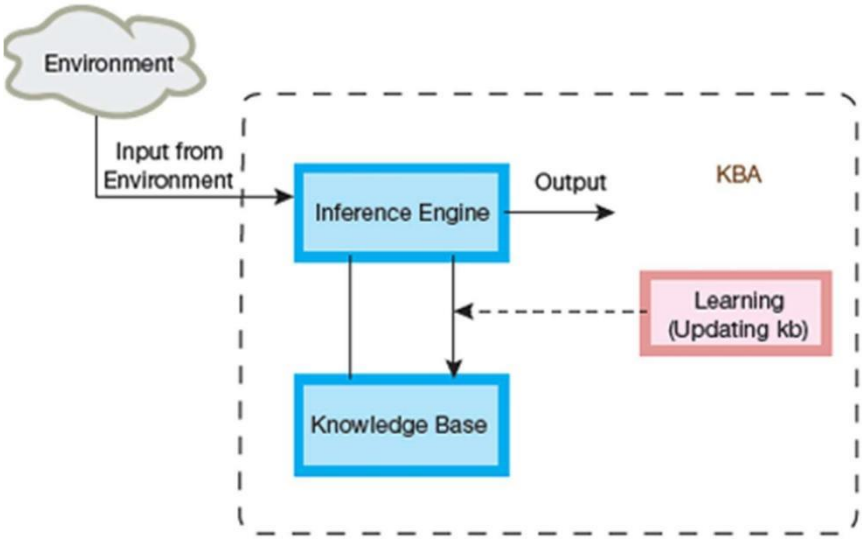
1. Representational accuracy, to ensure that the system represents all kinds of required knowledge.
2. Inferential adequacy to manipulate the representational structures to produce new knowledge corresponding to the existing structure.
3. Inferential efficiency to direct the inferential knowledge mechanism to generate appropriate results.
4. Acquisitional efficiency that helps the knowledge representation system to easily acquire new knowledge using automatic methods.

Knowledge Based Agents

The agents that mimic a human being's knowledge are known as knowledge-based intelligent agents. A knowledge-based agent, therefore, does the following:

1. It maintains an internal state of knowledge to represent states, actions, etc.
2. It deduces reasoning with that knowledge.
3. It updates their knowledge after observations by incorporating new precepts.
4. It takes actions accordingly

Architecture of knowledge-based agent



A knowledge-based agent (KBA) functions using two primary components: a knowledge base (KB) and an inference engine (IE). The KB stores factual information about the world, represented as sentences in a knowledge representation language. The IE processes inputs from the environment, interacts with the KB, and deduces new knowledge through logical reasoning methods such as forward or backward chaining.
The learning element of the KBA continually updates the KB with new information. The overall working involves three main steps: perceiving inputs from the environment, processing them through the inference engine, and making decisions using the stored knowledge.
KBA performs three key operations:

- TELL: Adds new information to the KB or updates existing knowledge.
- ASK: Queries the KB to decide which action to take.
- PERFORM: Executes the chosen action based on the inferred knowledge.

A knowledge-based agent (KBA) can be viewd as three distinct levels:

- Knowledge Level: Defines what the agent knows and what its goals are. This level specifies the agent's behavior by representing knowledge as sentences.
- Logical Level: Converts the knowledge into logical representations, enabling the agent to reason and deduce conclusions.
- Implementation Level: Represents the physical or operational phase where the agent performs actions based on the logic and knowledge derived from the previous levels.

In summary, the KBA first determines what it knows and aims for, then reasons using logic, and finally acts to achieve its goals.

A knowledge-based agent can be built using two main approaches:

- **Declarative Approach:** The agent starts with an empty knowledge base and is taught facts or sentences one by one until it can interact effectively with its environment.

- **Procedural Approach:** The agent's behavior is directly programmed using languages such as LISP or Prolog, specifying how it should act.
- **Hybrid Approach:** Combines both methods by converting declarative knowledge into efficient procedural code for practical performance.

Types of Knowledge

There are three main types of knowledge in a knowledge representation system, each explained below with examples:

1. Simple Relational Knowledge:
   Stores facts about objects in a tabular or relational form.
   Example:
   A student table showing relationships:

Name	Age	Course
Diya	20	B.Sc
Rahul	22	M.Sc

2. Inheritable Knowledge

   Represents information using a hierarchy of classes (general to specific) and uses the IS-A relationship.

   Example:
   A "Student" class has subclasses "Undergraduate Student" and "Postgraduate Student."
   Zinya → IS-A → Undergraduate Student
   Vidhi → IS-A → Postgraduate Student

3. Inferential Knowledge:
   Uses logic to represent and deduce new knowledge.
   Example:
   Statement 1: Diya is a student.
   Statement 2: All students are bright.
   From these, we can infer: Diya is bright.

This structure allows systems to store, classify, and infer knowledge effectively.